

**ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA CÔNG NGHỆ PHẦN MỀM**



**BÁO CÁO SEMINAR
MÔ HÌNH HÓA QUY TRÌNH CI/CD CỦA DEVOPS**

Môn học: Phương pháp mô hình hóa

GVHD: ThS. NGUYỄN CÔNG HOAN

SINH VIÊN THỰC HIỆN:

Nguyễn Đức Anh	24520083
Trần Nhật Duy	24520403
Nguyễn Đại Hưng	24520601
Châu Văn Phong	24521327

TP. HỒ CHÍ MINH, NGÀY 14 THÁNG 04 NĂM 2026

LỜI CẢM ƠN

Sự ra đời và phát triển của công nghệ phần mềm đòi hỏi tốc độ cập nhật liên tục để đáp ứng nhu cầu thị trường. Trong bối cảnh đó, sự chia tách giữa giai đoạn phát triển (Development) và vận hành (Operations) theo phương pháp truyền thống đã bộc lộ nhiều nhược điểm rõ rệt. Khái niệm DevOps và đặc biệt là quy trình tự động hóa CI/CD ra đời nhằm giải quyết bài toán này, giúp tối ưu hóa chu trình phát triển, mang lại các sản phẩm phần mềm nhanh hơn, an toàn và tin cậy hơn. Việc hiểu và mô hình hóa thành công quy trình CI/CD là một kỹ năng cốt lõi đối với sinh viên ngành Kỹ thuật Phần mềm trong kỷ nguyên hiện tại.

Trước tiên, nhóm xin bày tỏ lòng biết ơn sâu sắc đến **Thầy Nguyễn Công Hoan** — giảng viên phụ trách môn học *Phương Pháp Mô Hình Hóa* — vì những bài giảng hệ thống, súc tích và giàu tính ứng dụng trong suốt học kỳ vừa qua. Chính nền tảng lý thuyết vững chắc mà Thầy truyền đạt về các phương pháp mô hình hóa hiện đại đã tạo tiền đề để nhóm tiếp cận và triển khai đề tài “*Mô hình hóa quy trình CI/CD trong DevOps*” một cách có hệ thống và khoa học.

Bên cạnh đó, những phản hồi chuyên môn và định hướng mà Thầy cung cấp trong quá trình học tập trên lớp đã giúp nhóm xác định rõ phạm vi vấn đề, lựa chọn công cụ mô hình hóa phù hợp, cũng như củng cố tính chặt chẽ trong luận điểm trình bày. Những góc nhìn thực tiễn từ Thầy không chỉ nâng cao chất lượng học thuật của báo cáo, mà còn bồi dưỡng tư duy phản biện và năng lực nghiên cứu độc lập cho từng thành viên trong nhóm.

Tuy nhiên, do giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn tiếp xúc với các hệ thống lớn chưa nhiều, bài báo cáo của nhóm chắc chắn không tránh khỏi những thiếu sót trong quá trình đánh giá và thiết kế mô hình. Chúng em rất mong nhận được những phản hồi và góp ý từ Thầy cùng các bạn sinh viên để tiếp tục hoàn thiện sản phẩm, đồng thời tích lũy thêm hành trang cho những dự án nghiên cứu và seminar tiếp theo.

Chúng em xin chân thành cảm ơn Thầy!

TP. Hồ Chí Minh, ngày 14 tháng 04 năm 2026

Nhóm sinh viên thực hiện

Nguyễn Đức Anh

Trần Nhật Duy

Nguyễn Đại Hưng

Châu Văn Phong

MỤC LỤC

LỜI CẢM ƠN	1
DANH MỤC HÌNH ẢNH	3
I. MỞ ĐẦU	4
1. Giới thiệu đề tài	4
2. Mục tiêu nghiên cứu	4
3. Đối tượng và phạm vi nghiên cứu	5
II. Cơ sở lý thuyết về hệ thống tự động hóa CI/CD	6
1. Nguồn gốc và sự hình thành của DevOps	6
2. Các khái niệm cốt lõi	7
a. Tích hợp liên tục (Continuous Integration - CI)	7
b. Phân phối liên tục (Continuous Delivery - CD)	9
c. Triển khai liên tục (Continuous Deployment - CD)	9
3. Lợi ích và thách thức khi mô hình hóa quy trình tự động	10
III. CÁC GIAI ĐOẠN TRONG QUY TRÌNH CI/CD	11
1. Cấu trúc tuyến tính của một quy trình	11
2. Chi tiết 5 giai đoạn cốt lõi của quy trình CI/CD	12
a. Giai đoạn 1: Lập trình và Kích hoạt (Code & Trigger)	12
b. Giai đoạn 2: Biên dịch và Đóng gói (Build)	12
c. Giai đoạn 3: Kiểm thử tự động (Automated Test)	13
d. Giai đoạn 4: Đóng gói phiên bản (Release)	13
e. Giai đoạn 5: Triển khai và Giám sát (Deploy & Monitor)	13
3. Các chiến lược triển khai không gián đoạn (Zero-Downtime Deployment - ZDD)	14
a. Chiến lược Blue-Green Deployment	14
b. Chiến lược Canary Release	15
c. Chiến lược Rolling Update	16
d. Bảng so sánh các chiến lược	17
IV. MÔ HÌNH HÓA QUY TRÌNH CI/CD CỦA DEVOPS	17
V. KẾT LUẬN	18
1. Thành tựu đạt được	18
2. Hạn chế của đề tài	18
3. Hướng phát triển trong tương lai	19
TÀI LIỆU THAM KHẢO	20

DANH MỤC HÌNH ẢNH

Hình 1.	Mối tương quan và trọng tâm khác biệt giữa Agile, CI/CD và DevOps . .	7
Hình 2.	Mô hình vòng đời DevOps và các giai đoạn trong quy trình CI/CD	7
Hình 3.	Sơ đồ các giai đoạn trong một đường ống CI/CD (CI/CD Pipeline) tiêu chuẩn.	8
Hình 4.	Quy trình các bước trong giai đoạn Tích hợp liên tục (Continuous Integration)	8
Hình 5.	Các thành phần cốt lõi của chu trình Phân phối liên tục (Continuous Delivery)	9
Hình 6.	Sự khác biệt về tính tự động giữa Continuous Delivery và Continuous Deployment	10
Hình 7.	Mô hình kiến trúc chuyển hướng luồng truy cập trong Blue-Green Deployment	14
Hình 8.	Mô hình phân luồng lưu lượng truy cập trong Canary Release	15
Hình 9.	Mô hình hóa luồng công việc CI/CD và các chiến lược triển khai thành phẩm	17
Hình 10.	Luồng dữ liệu từ môi trường phát triển cục bộ đến triển khai thực tế qua Pipeline	18

I. MỞ ĐẦU

1. Giới thiệu đề tài

Trong kỷ nguyên số hóa hiện đại, tốc độ đưa sản phẩm phần mềm ra thị trường (time-to-market) đã trở thành một trong những chỉ số cạnh tranh mang tính sống còn đối với các tổ chức công nghệ. Tuy nhiên, sự phát triển thần tốc của nhu cầu thị trường đã làm lộ rõ sự chênh lệch sâu sắc giữa phương pháp phát triển phần mềm truyền thống và thực tiễn vận hành. Trong các mô hình tổ chức kiểu silo (phân mảnh), sự chia tách giữa nhóm Phát triển (Development - Dev) và nhóm Vận hành (Operations - Ops) đã tạo ra một ranh giới nhân tạo, được Kim và cộng sự gọi là “bức tường confuse” (Wall of Confusion) [1]. Trong bối cảnh đó, nhóm Dev bị áp lực bởi metric tốc độ đưa ra tính năng mới, trong khi Ops lại bị trói buộc bởi metric ổn định hệ thống. Sự xung đột mục tiêu này dẫn đến một vòng luẩn quẩn suy thoái: mã nguồn được “ném” qua tường một cách thiếu kiểm soát, gây ra sự cố trên môi trường thực tế (Production), và kết quả tất yếu là quá trình phát hành bị trì hoãn nghiêm trọng [2].

Nhằm giải quyết triệt để bài toán hệ thống này, triết lý DevOps ra đời như một sự tiến hóa tất yếu dựa trên nền tảng của Agile và Lean. Khác với Agile chỉ tập trung vào chu trình phát triển, DevOps mở rộng chuỗi giá trị đến khi phần mềm thực sự vận hành ổn định [3, 4]. Động cơ thực thi cốt lõi của DevOps được biểu hiện thông qua hệ thống tự động hóa CI/CD (Continuous Integration/Continuous Delivery hoặc Continuous Deployment). Theo định nghĩa của Humble và Farley, CI/CD không đơn thuần là một tập hợp các công cụ, mà là một phương pháp luận giúp “xây dựng chất lượng vào trong” (build quality in) phần mềm ngay từ những giai đoạn đầu tiên, đảm bảo mã nguồn luôn ở trạng thái có thể triển khai được [5]. Nhờ đó, các tổ chức tiên phong như Netflix hay Etsy có thể thực hiện hàng trăm lần triển khai mỗi ngày, biến việc phát hành phần mềm từ một sự kiện căng thẳng thành một hoạt động thường nhật mang tính tự động cao [6].

Mặc dù mang lại lợi ích vượt trội, nhưng bản thân quy trình CI/CD trong thực tế là một hệ thống phân tán cực kỳ phức tạp. Nó bao gồm hàng loạt công cụ (Git, Jenkins, Docker, Kubernetes) tương tác với nhau qua các API, webhook với các luồng dữ liệu (data flow) và luồng điều khiển (control flow) đan xen chặt chẽ [7]. Đặc biệt, ở giai đoạn cuối của pipeline, việc tích hợp các chiến lược triển khai không gián đoạn (Zero-Downtime Deployment - ZDD) như Blue-Green, Canary Release hay Rolling Update càng làm tăng tính phức tạp của hệ thống, đòi hỏi khả năng đồng bộ hóa trạng thái và xử lý ngoại lệ chính xác [8, 9, 10].

Trong bối cảnh môn học Phương pháp Mô hình hóa, việc đề một hệ thống phức tạp như CI/CD chỉ tồn tại dưới dạng các kịch bản cấu hình (script) hoặc tài liệu văn bản là một thiếu sót lớn về mặt kỹ thuật. Thiếu vắng một “ngôn ngữ chung” trực quan dẫn đến sự mơ hồ trong giao tiếp giữa các bên liên quan (Stakeholders), khiến việc đánh giá rủi ro kiến trúc và bảo trì hệ thống trở nên vô cùng khó khăn [11]. Do đó, việc nghiên cứu và áp dụng các ngôn ngữ mô hình hóa chuẩn công nghiệp để đặc tả hình thức (formal specification) quy trình CI/CD là một đòi hỏi cấp thiết. Đây chính là động lực cốt lõi thúc đẩy nhóm quyết định lựa chọn đề tài: **“Mô hình hóa quy trình CI/CD của DevOps”**, nhằm thu hẹp khoảng cách giữa lý thuyết mô hình hóa và thực tiễn tự động hóa hiện đại.

2. Mục tiêu nghiên cứu

Xuất phát từ bối cảnh thực tiễn nêu trên, bài báo cáo này không chỉ dừng lại ở việc mô tả bề nổi của quy trình CI/CD, mà hướng tới việc xây dựng một hệ thống mô hình mang tính khái quát hóa cao, đủ sức làm “bản thiết kế” (blueprint) cho việc triển khai hệ thống thực. Mục tiêu nghiên cứu của đề tài được chia thành hai cấp độ: mục tiêu tổng quát và các mục tiêu cụ thể.

Mục tiêu tổng quát: Nghiên cứu, phân tích và áp dụng các phương pháp mô hình hóa (đặc

biệt là ngôn ngữ UML - Unified Modeling Language) để đặc tả một cách toàn diện, chính xác và chặt chẽ về mặt logic đối với quy trình tự động hóa CI/CD trong hệ sinh thái DevOps, bao gồm cả các chiến lược triển khai không gián đoạn ở giai đoạn cuối.

Các mục tiêu cụ thể:

1. **Phân tích và giải phóng bề mặt kiến trúc của quy trình CI/CD:** Nghiên cứu sâu cơ sở lý thuyết về CI/CD, từ đó bóc tách hệ thống phân tán này thành các thành phần chức năng độc lập. Xác định rõ ranh giới trách nhiệm giữa Tích hợp liên tục (CI) với hai khía cạnh của Phân phối/Triển khai liên tục (CD) dựa trên lăng kính của các khung lý thuyết chuẩn [5, 12].
2. **Mô hình hóa cấu trúc tĩnh (Structural Modeling):** Sử dụng các sơ đồ lớp (Class Diagram) và sơ đồ thành phần (Component Diagram) để xây dựng mô hình kiến trúc của đường ống CI/CD. Mục tiêu là trực quan hóa các thực thể phần mềm, cơ sở hạ tầng (ví dụ: Kubernetes Clusters, AWS RDS [13]) và các phụ thuộc giao tiếp giữa các công cụ trong chuỗi toolchain, từ đó giải quyết bài toán mơ hồ trong đặc tả hệ thống [11].
3. **Mô hình hóa hành vi và luồng công việc động (Behavioral Modeling):** Áp dụng sơ đồ hoạt động (Activity Diagram) và sơ đồ trình tự (Sequence Diagram) để mô phỏng vòng đời tuyến tính của pipeline. Mô hình này phải phản ánh chính xác các “chốt chặn” kiểm soát chất lượng (quality gates), cơ chế trigger tự động qua webhook, và đặc biệt là phải bắt được các vòng lặp phản hồi (feedback loops) phức tạp – ví dụ như luồng xử lý khi một bài kiểm thử (test) thất bại [7].
4. **Mô hình hóa các chiến lược triển khai Zero-Downtime Deployment (ZDD):** Đây là mục tiêu mang tính kỹ thuật sâu nhất của bài báo cáo. Xây dựng các mô hình trạng thái (State Machine Diagram) và luồng hoạt động cho ba chiến lược ZDD phổ biến: Blue-Green Deployment, Canary Release và Rolling Update [10]. Mục tiêu là làm rõ cơ chế chuyển đổi luồng truy cập (traffic routing), trạng thái tương thích ngược của cơ sở dữ liệu, và quy trình rollback trong từng kịch bản cụ thể.
5. **Đánh giá mức độ trừu tượng hóa (Level of Abstraction):** Đưa ra các luận chứng phản biện về việc lựa chọn điểm cân bằng tối ưu trong mô hình hóa CI/CD: đủ trừu tượng để không bị phụ thuộc vào công nghệ cụ thể (technology-agnostic), nhưng lại đủ chi tiết để giữ được giá trị kỹ thuật, tránh rơi vào cạm bẫy của một mô hình quá cứng nhắc hoặc quá chung chung [11].

3. Đối tượng và phạm vi nghiên cứu

Để đảm bảo tính khả thi và độ sâu cho một báo cáo seminar, việc xác lập ranh giới nghiên cứu là bước đi mang tính quyết định, giúp tập trung nguồn lực phân tích vào lõi của vấn đề mà không bị lan man.

Đối tượng nghiên cứu: Đối tượng nghiên cứu trọng tâm của bài báo cáo là **quy trình tự động hóa CI/CD (CI/CD Pipeline)** dưới lăng kính của khoa học mô hình hóa hệ thống. Đối tượng này bao gồm hai lớp cấu trúc chính:

- *Lớp luồng nghiệp vụ cốt lõi:* Chuỗi các giai đoạn từ lúc Developer push mã nguồn (commit) lên hệ thống quản lý phiên bản, trải qua quá trình biên dịch (build), kiểm thử tự động (test), đóng gói (packaging) cho đến khi tạo ra sản phẩm thực thi (artifact) [5].
- *Lớp chiến lược triển khai phân tán:* Các cơ chế điều hướng lưu lượng mạng và cập nhật trạng thái máy chủ ở giai đoạn cuối của pipeline, cụ thể là các chiến lược Blue-Green, Canary Release và Rolling Update [8, 9, 10].

Phạm vi nghiên cứu:

- **Phạm vi về mặt không gian (Spatial Scope):** Nghiên cứu giới hạn trong ranh giới của đường ống CI/CD và hệ thống cơ sở hạ tầng triển khai (Deployment Infrastructure). Bài báo cáo **không** đi sâu vào mô hình hóa các quy trình quản lý dự án (như Scrum/Kanban), các chiến lược phát triển tính năng (Feature Flagging), hay các khía cạnh liên quan đến văn hóa tổ chức con người của DevOps [1].
- **Phạm vi về mặt công nghệ (Technological Scope):** Mặc dù trong thực tế, mỗi tổ chức sử dụng một bộ công cụ (toolchain) khác nhau (ví dụ: GitLab CI thay vì Jenkins, ArgoCD thay vì Spinnaker), nghiên cứu này áp dụng nguyên tắc *technology-agnostic*. Mô hình hóa sẽ không phụ thuộc vào cú pháp của một công cụ cụ thể, mà tập trung vào các thực thể logic (Ví dụ: “CI Server”, “Container Registry”, “Load Balancer”, “Orchestrator”). Việc đưa ra ví dụ về Kubernetes hay AWS RDS Blue/Green [13] chỉ đóng vai trò là minh chứng thực tiễn (use-case) để làm rõ mô hình, không phải là đối tượng để cấu hình chi tiết.
- **Phạm vi về mặt chiến lược triển khai (Deployment Strategy Scope):** Trong mảng ZDD, phạm vi phân tích bị giới hạn ở ba chiến lược kinh điển nhất hiện nay. Nhóm nghiên cứu sẽ không mở rộng đến các chiến lược cấp cao hơn hoặc lai ghép phức tạp như A/B Testing (vì thiên về đánh giá metric kinh doanh hơn là kỹ thuật hệ thống), hay Shadow Deployment (do bản chất không phục vụ traffic thật, làm sai lệch mô hình trạng thái).
- **Phạm vi về mặt ngôn ngữ mô hình hóa (Modeling Language Scope):** Bài báo cáo sử dụng bộ tiêu chuẩn UML 2.x làm công cụ mô hình hóa chính [11]. Sẽ không sử dụng các ngôn ngữ đặc thù khác như BPMN (Business Process Model and Notation - do thiên về quy trình nghiệp vụ kinh doanh hơn là kiến trúc phần mềm) hay SysML (do quá nặng nề cho bài toán CI/CD phần mềm thuần túy).

II. Cơ sở lý thuyết về hệ thống tự động hóa CI/CD

1. Nguồn gốc và sự hình thành của DevOps

Trước khi đi sâu vào các khái niệm kỹ thuật của CI/CD, việc hiểu nguồn gốc hình thành DevOps là điều kiện tiên quyết để nhận thức đầy đủ về bản chất của hệ thống tự động hóa hiện đại. Trong các tổ chức phát triển phần mềm truyền thống, việc vận hành theo mô hình thác nước (Waterfall) kết hợp với cơ cấu tổ chức kiểu silo (phân mảnh) đã tạo ra một “bức tường confuse” – ranh giới nhân tạo giữa nhóm Phát triển (Dev) và nhóm Vận hành (Ops) [1]. Trong bối cảnh này, Dev được đo lường bằng tốc độ đưa ra tính năng mới, trong khi Ops lại bị đánh giá qua lăng kính ổn định và an toàn của hệ thống. Sự xung đột về KPI này tạo ra một vòng luẩn quẩn suy thoái: Dev “ném” mã nguồn qua tường, Ops nhận được các bản thay đổi kém chất lượng dẫn đến sự cố, và kết quả là quá trình phát hành bị trì hoãn, chất lượng giảm sút [2].

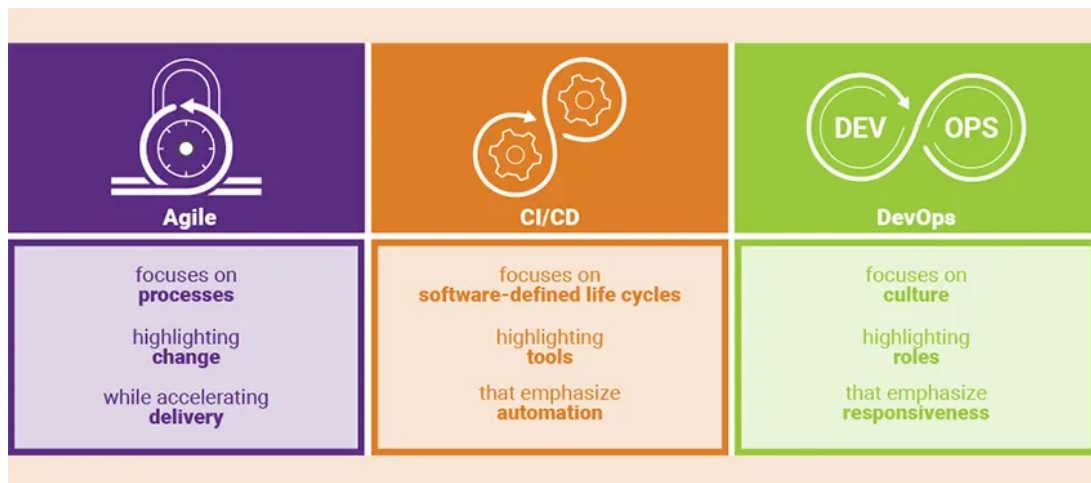


Figure 1: Môi trường quan và trọng tâm khác biệt giữa Agile, CI/CD và DevOps

Khái niệm "DevOps" chính thức được Patrick Debois đặt tên vào năm 2009 tại hội nghị DevOpsDays đầu tiên ở Ghent, Bỉ, đánh dấu sự khởi đầu của một phong trào văn hóa rộng lớn [3]. Tuy nhiên, DevOps không phải là một công cụ hay một đội ngũ mới, mà là sự tiến hóa tất yếu dựa trên nền tảng của Agile và Lean. Nếu Agile tập trung vào việc phát triển đúng thứ cần thiết thông qua các vòng lặp phản hồi ngắn, thì DevOps mở rộng chuỗi giá trị này đến lúc phần mềm chạy thực tế trên môi trường Production thông qua việc tự động hóa và vận hành liên tục [4]. Cuốn tiểu thuyết *The Phoenix Project* đã khắc họa rõ nét cuộc khủng hoảng của mô hình cũ, đồng thời chỉ ra rằng giải pháp cốt lõi không nằm ở việc mua sắm công cụ mới, mà ở việc phá vỡ sự cô lập giữa các nhóm chức năng, thiết lập vòng phản hồi nhanh và chia sẻ trách nhiệm chung [2]. Tóm lại, DevOps được định nghĩa bằng công thức: **DevOps = Văn hóa + Quy trình + Tự động hóa**, trong đó tự động hóa CI/CD chính là động cơ thực thi triết lý này.

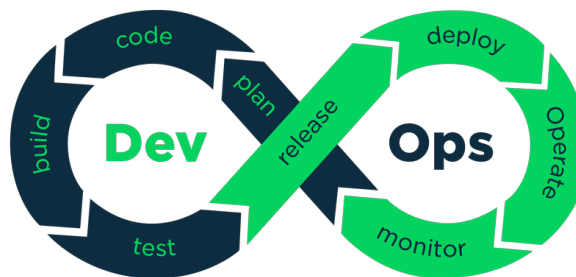


Figure 2: Mô hình vòng đời DevOps và các giai đoạn trong quy trình CI/CD

2. Các khái niệm cốt lõi

Hệ thống tự động hóa trong DevOps được xây dựng dựa trên ba trụ cột kỹ thuật cốt lõi: Continuous Integration (CI) và hai khía cạnh của Continuous Delivery/Deployment (CD). Dù thường được viết gộp là CI/CD, mỗi khái niệm mang một ý nghĩa và ranh giới trách nhiệm riêng biệt.

a. Tích hợp liên tục (Continuous Integration - CI)

Tích hợp liên tục (CI) là thực hành mà trong đó các nhà phát triển thường xuyên (nhiều lần trong ngày) hợp nhất mã nguồn của họ vào nhánh dùng chung (main/trunk). Mỗi lần tích hợp đều được kích hoạt tự động thông qua webhook, đi kèm với quá trình biên dịch (build) và chạy kiểm thử (test) tự động [5].

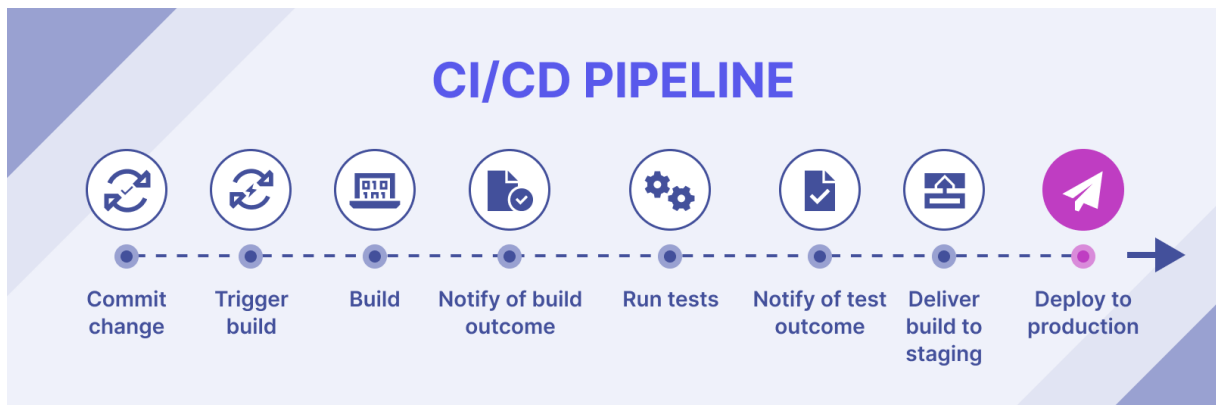


Figure 3: Sơ đồ các giai đoạn trong một đường ống CI/CD (CI/CD Pipeline) tiêu chuẩn.

Theo Humble và Farley, mục tiêu tối thượng của CI không chỉ là tìm lỗi, mà là *”xây dựng chất lượng vào trong”* (build quality in) và phát hiện lỗi ngay khi chúng vừa được tạo ra, khi chi phí sửa chữa còn thấp nhất [5]. Khi thiếu CI, các nhánh (branch) chỉ được hợp nhất vào giai đoạn cuối của dự án, dẫn đến xung đột mã nguồn (merge conflict) không lồ và làm hỏng toàn bộ bản build. Quy trình CI cơ bản bắt đầu khi Developer push code lên hệ thống quản lý phiên bản (Git). CI server sẽ lập tức trigger thực hiện các bước: biên dịch mã nguồn thành các artifact thực thi (ví dụ: file .jar, .war), chạy các bài kiểm thử đơn vị (unit test) và phân tích tĩnh mã nguồn (lint/static analysis). Nguyên tắc bất di bất dịch của CI là: *nếu bất kỳ bước nào thất bại (test fail), pipeline phải dừng lại ngay lập tức* [5]. Điều này đảm bảo mã nguồn trên nhánh chính luôn ở trạng thái có thể triển khai được, tạo tiền đề vững chắc cho các giai đoạn tiếp theo.

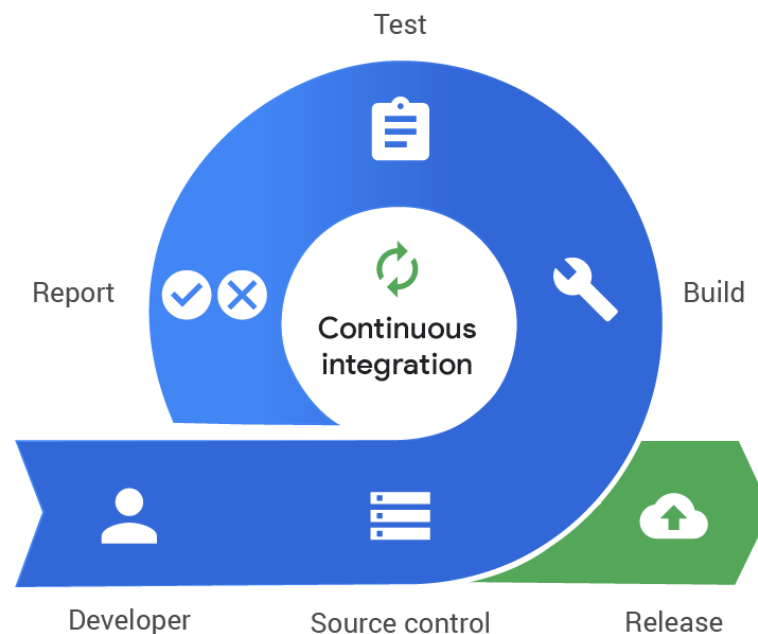


Figure 4: Quy trình các bước trong giai đoạn Tích hợp liên tục (Continuous Integration)

b. Phân phối liên tục (Continuous Delivery - CD)

Nếu CI đảm bảo mã nguồn được tích hợp an toàn, thì Phân phối liên tục (Continuous Delivery) là thực hành đảm bảo mã nguồn đó *luôn sẵn sàng* để được phát hành ra tay người dùng cuối [5]. Sự khác biệt lớn nhất giữa CI và CD đầu tiên nằm ở điểm kết thúc của pipeline: CI kết thúc khi tạo ra artifact, trong khi CD mở rộng pipeline xuyên suốt từ lúc commit cho đến việc triển khai lên các môi trường (Staging, Production).

Đặc điểm định nghĩa của Continuous Delivery là bước chuyển giao cuối cùng lên môi trường Production vẫn **yêu cầu sự can thiệp thủ công của con người** (manual approval). Nghĩa là, mọi bước từ đóng gói (packaging) mã nguồn vào các container (như Docker Image) cho đến việc chạy kiểm thử tích hợp, kiểm thử chấp nhận (acceptance test) và quét bảo mật đều được tự động hóa hoàn toàn. Tuy nhiên, trước khi bấm nút phát hành chính thức cho người dùng, một quy trình phê duyệt (gate) được thực hiện để đảm bảo tính đúng đắn về mặt kinh doanh [1]. Cách tiếp cận này giúp tổ chức duy trì được tốc độ tự động hóa cao nhưng vẫn kiểm soát được rủi ro chiến lược, rất phù hợp với các hệ thống đòi hỏi tuân thủ nghiêm ngặt (compliance) hoặc có tính chất kinh doanh đặc thù.



Figure 5: Các thành phần cốt lõi của chu trình Phân phối liên tục (Continuous Delivery)

c. Triển khai liên tục (Continuous Deployment - CD)

Triển khai liên tục (Continuous Deployment) là sự mở rộng tối đa của Continuous Delivery, ở đó **toàn bộ quy trình từ commit đến Production diễn ra hoàn toàn tự động, không có bất kỳ sự can thiệp thủ công nào** [3]. Trong mô hình này, nếu mã nguồn vượt qua tất cả các giai đoạn kiểm thử tự động trong pipeline CI/CD, nó sẽ tự động được đưa lên môi trường Production và phục vụ người dùng cuối.

Để một hệ thống có thể áp dụng Continuous Deployment một cách an toàn, nó đòi hỏi một nền tảng kỹ thuật cực kỳ maturity. Cụ thể, hệ thống phải sở hữu bộ kiểm thử tự động toàn diện (test coverage cao), khả năng triển khai không gián đoạn (Zero-Downtime Deployment như Blue-Green hoặc Canary Release) để giảm thiểu bán kính ảnh hưởng (blast radius) khi có lỗi, và hệ thống giám sát (monitoring/telemetry) tinh vi để phát hiện bất thường và thực hiện rollback tự động ngay lập tức [1]. Nhờ đó, các tổ chức áp dụng Continuous Deployment (như Etsy, Netflix) có thể thực hiện hàng chục, hàng trăm lần triển khai mỗi ngày, biến việc phát hành phần mềm từ một sự kiện căng thẳng thành một hoạt động thường nhật, mang lại lợi thế cạnh tranh không lồ về thời gian ra thị trường (time-to-market) [3].

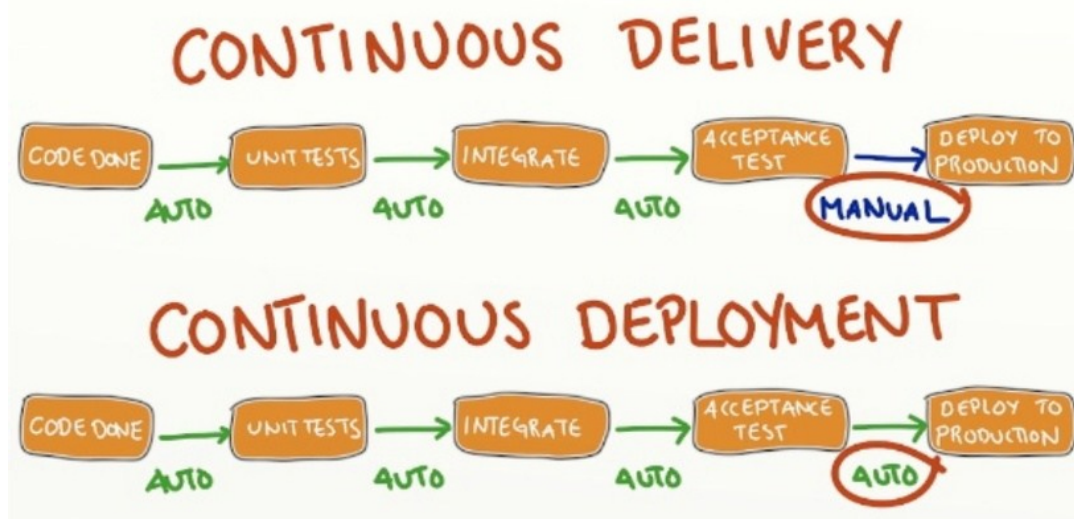


Figure 6: Sự khác biệt về tính tự động giữa Continuous Delivery và Continuous Deployment

3. Lợi ích và thách thức khi mô hình hóa quy trình tự động

Trong bối cảnh môn học Phương pháp mô hình hóa, việc mô hình hóa quy trình CI/CD không chỉ đơn thuần là vẽ lại các sơ đồ khối, mà là quá trình đặc tả hình thức (formal specification) một hệ thống phần mềm phân tán và phức tạp. Việc vận dụng các ngôn ngữ mô hình hóa chuẩn công nghiệp như UML (Unified Modeling Language) mang lại những giá trị lý luận và thực tiễn sâu sắc [11].

Về mặt lợi ích, mô hình hóa giúp *trực quan hóa* và *xóa bỏ sự mơ hồ* (ambiguity) trong hệ thống CI/CD. Một quy trình tự động bao gồm hàng loạt công cụ khác nhau (Git, Jenkins, Docker, Kubernetes) giao tiếp với nhau qua các API và webhook. Việc sử dụng các sơ đồ trình tự (Sequence Diagram) hoặc đồ hoạt động (Activity Diagram) giúp các bên liên quan (Stakeholders) có một "ngôn ngữ chung" để hiểu luồng dữ liệu (data flow) và luồng điều khiển (control flow) giữa các giai đoạn [11]. Thứ hai, mô hình hóa phát hiện ra các điểm nút thắt (bottlenecks) và rủi ro kiến trúc trước khi viết mã cấu hình thực tế. Ví dụ, khi mô hình hóa, nhóm có thể nhận ra việc thiết kế pipeline chạy tuần tự sẽ làm tăng thời gian tích hợp, từ đó đưa ra quyết định mô hình hóa lại dưới dạng các luồng song song (parallel stage). Cuối cùng, mô hình đóng vai trò là tài liệu "live documentation", phục vụ cho việc bảo trì, mở rộng và bàn giao kiến thức trong một lĩnh vực mà sự phụ thuộc vào tool là rất lớn.

Mặc dù vậy, việc mô hình hóa quy trình CI/CD cũng đối mặt với nhiều thách thức đáng kể. Thách thức lớn nhất là *sự mất cân bằng giữa tính tĩnh của mô hình và tính động của hệ thống thực*. Các pipeline CI/CD trong thực tế mang tính trạng thái (stateful) và có tính phản hồi vòng lặp (feedback loop) phức tạp – ví dụ, khi một giai đoạn Test thất bại, pipeline có thể trigger lại từ giai đoạn Build hoặc gửi thông báo phức tạp đến nhiều hệ thống khác nhau. Việc bắt sát hoàn

toàn các logic rẽ nhánh và xử lý ngoại lệ (exception handling) này bằng các mô hình tĩnh như UML hay BPMN là rất khó khăn và dễ làm cho sơ đồ trở nên rối rắm [11]. Thứ hai, thách thức về mức độ trừu tượng hóa (level of abstraction). Nếu mô hình quá chi tiết (mô phỏng từng câu lệnh script), nó sẽ trở nên cứng nhắc và nhanh chóng lỗi thời khi công cụ thay đổi. Ngược lại, nếu mô hình quá trừu tượng, nó sẽ mất đi giá trị kỹ thuật, không khác gì một lời giới thiệu văn bản. Do đó, bài toán cốt lõi khi mô hình hóa quy trình CI/CD là phải tìm ra điểm cân bằng tối ưu: tạo ra một mô hình đủ trừu tượng để chịu sự thay đổi của công nghệ (technology-agnostic), nhưng lại đủ chi tiết để có thể dùng làm cơ sở (blueprint) triển khai hệ thống thực.

III. CÁC GIAI ĐOẠN TRONG QUY TRÌNH CI/CD

1. Cấu trúc tuyến tính của một quy trình

Về bản chất, cấu trúc tuyến tính của một quy trình CI/CD là một chuỗi các bước thiết lập sẵn (*pipeline*) mà các nhà phát triển phải tuân thủ để phát hành một phiên bản phần mềm mới. Theo định nghĩa từ **Red Hat**: *"A continuous integration and continuous deployment (CI/CD) pipeline is a series of established steps that developers must follow in order to deliver a new version of software. CI/CD pipelines are a practice focused on improving software delivery throughout the software development life cycle via automation."*

Cấu trúc này được vận hành dựa trên một luồng công việc nối tiếp nhau một cách chặt chẽ:

- **Tích hợp liên tục (CI):** Đây là giai đoạn các thành viên trong nhóm hợp nhất các thay đổi mã nguồn vào một nhánh chung với tần suất ít nhất hàng ngày. Theo **Martin Fowler** [7], mỗi sự tích hợp này được xác nhận bởi một hệ thống xây dựng tự động (*automated build*) bao gồm cả việc kiểm thử để phát hiện lỗi tích hợp càng nhanh càng tốt. Phương pháp này giúp giảm thiểu rủi ro chậm trễ và nỗ lực tích hợp, đồng thời duy trì một nền tảng mã nguồn khỏe mạnh để phát triển các tính năng mới.
- **Phân phối liên tục (Continuous Delivery):** Sau khi vượt qua giai đoạn CI, mã nguồn được chuyển đến môi trường *Staging* để thực hiện các bài kiểm tra tích hợp và kiểm tra chấp nhận từ người dùng.
- **Triển khai liên tục (Continuous Deployment):** Đây là điểm cuối của trục tuyến tính, nơi các rào cản thủ công được loại bỏ. Mã nguồn đạt chuẩn sẽ tự động được đẩy trực tiếp lên môi trường *Production*.
- Nếu CI là bước khởi đầu thì CD chính là chặng đường cuối cùng để đưa phần mềm đến tay người dùng. Theo **Martin Fowler** [12], việc mã nguồn được tích hợp thành công vẫn chưa đủ để nó thực hiện nhiệm vụ của mình. CD giải quyết "chặng cuối" (*last mile*) này bằng cách xây dựng các *deployment pipelines* nhằm chuyển đổi mã đã tích hợp thành phần mềm chạy trên môi trường thực tế (*production software*). Quá trình này giúp việc phát hành phần mềm trở thành một hoạt động thường nhật, giảm thiểu rủi ro và áp lực cho đội ngũ phát triển.

Tính chất tuyến tính này đảm bảo rằng không có bất kỳ thay đổi nào được phát hành mà không đi qua đầy đủ các "chốt chặn" kiểm soát chất lượng, giúp tối ưu hóa tốc độ bàn giao nhưng vẫn giữ vững sự ổn định tuyệt đối cho hệ thống.

2. Chi tiết 5 giai đoạn cốt lõi của quy trình CI/CD

a. Giai đoạn 1: Lập trình và Kích hoạt (Code & Trigger)

Giai đoạn khởi đầu của một quy trình CI/CD tập trung vào việc quản lý sự thay đổi mã nguồn và tự động hóa việc bắt đầu chuỗi cung ứng phần mềm.

- **Thực thi lập trình và Quản lý phiên bản:** Lập trình viên viết mã nguồn cho các tính năng mới và thực hiện lệnh *push* lên kho lưu trữ chung như GitHub hoặc GitLab. Điểm cốt lõi của bước này là việc tích hợp thường xuyên, nơi các thành viên trong nhóm hợp nhất các thay đổi của họ vào nhánh chính ít nhất hàng ngày để tránh xung đột mã nguồn nghiêm trọng.[7]
- **Cơ chế Kích hoạt (Trigger):** Để duy trì tính liên tục, quy trình sử dụng cơ chế **Webhook**. Ngay khi kho lưu trữ nhận diện được sự thay đổi (như một sự kiện *push* hoặc *merge request*), Webhook sẽ phát tín hiệu để tự động kích hoạt đường ống (*pipeline*) mà không cần bất kỳ sự can thiệp thủ công nào. Điều này hiện thực hóa việc cải thiện vòng đời phần mềm thông qua tự động hóa.
- **Chiến lược phân nhánh (Branching Strategy):** Để kiểm soát tính ổn định tại điểm nút này, đội ngũ phải áp dụng các quy tắc quản lý nhánh như:
 - *Gitflow*: Phân chia rõ rệt giữa các nhánh tính năng (*feature*), phát triển (*develop*) và sản phẩm (*master/main*).
 - *Trunk-based Development*: Tập trung vào việc tích hợp các thay đổi nhỏ trực tiếp vào nhánh chính để tăng tốc độ phản hồi.

Các chiến lược này đóng vai trò là bộ quy tắc xác định cách thức tương tác, đặt tên và kiểm soát quyền hạn đối với các nhánh quan trọng, đảm bảo luồng tuyến tính của quy trình luôn được giữ vững.

b. Giai đoạn 2: Biên dịch và Đóng gói (Build)

Mục tiêu của giai đoạn này là chuyển đổi mã nguồn từ ngôn ngữ bậc cao thành các đơn vị có thể triển khai thực tế thông qua môi trường cô lập.

- **Tự động hóa biên dịch:** Hệ thống tự động cấp phát các máy ảo (*Virtual Machines*) hoặc *Containers* để thực hiện việc biên dịch mã nguồn. Quá trình này giúp loại bỏ sự phụ thuộc vào cấu hình cá nhân của lập trình viên, đảm bảo rằng "mọi bản build đều được thực hiện trong một môi trường sạch và nhất quán" [5].
- **Quản lý thành phần phụ thuộc (Dependency Management):** Pipeline tự động tải và kiểm tra các thư viện liên quan (như các gói từ NuGet, NPM, hoặc Maven). Việc kiểm soát chặt chẽ các phiên bản thư viện ở bước này giúp ngăn ngừa các lỗi bảo mật và xung đột phiên bản trong môi trường thực tế.
- **Tạo bản đóng gói (Artifact Creation):** Kết quả của giai đoạn này là các tệp tin thực thi hoặc hình ảnh (*images*) đóng gói sẵn. Các thực thể này được lưu trữ vào kho lưu trữ tập trung (*Artifact Repository*), cho phép cùng một bản build duy nhất được sử dụng xuyên suốt từ khâu kiểm thử đến triển khai, tuân thủ nguyên tắc "xây dựng một lần, triển khai mọi nơi" [5].

c. Giai đoạn 3: Kiểm thử tự động (Automated Test)

Đây là giai đoạn cốt lõi để thực hiện triết lý "thất bại sớm" (*fail-fast*), giúp phát hiện và ngăn chặn các lỗi kỹ thuật trước khi chúng tiến sâu hơn vào luồng phát hành.

- **Kiểm thử đa tầng:** Hệ thống thực thi chuỗi kịch bản kiểm thử từ thấp đến cao, bao gồm kiểm thử đơn vị (*Unit Test*) để xác định tính đúng đắn của các hàm mã nguồn và kiểm thử tích hợp (*Integration Test*) nhằm đảm bảo sự tương tác mượt mà giữa các module hoặc dịch vụ [7, 5].
- **Phân tích mã nguồn tĩnh (SAST):** Các công cụ tự động được sử dụng để quét mã nguồn nhằm phát hiện lỗ hổng bảo mật, lỗi logic tiềm ẩn hoặc vi phạm tiêu chuẩn lập trình mà không cần thực thi chương trình.
- **Kiểm thử hồi quy (Regression Test):** Pipeline tự động kiểm tra lại các tính năng hiện có để đảm bảo rằng các thay đổi mới không gây ra tác động tiêu cực đến sự ổn định của hệ thống cũ [5].
- **Phản hồi tức thời:** Nếu bất kỳ bài kiểm tra nào thất bại, quy trình sẽ ngay lập tức dừng lại và thông báo trạng thái lỗi cho đội ngũ phát triển. Điều này giúp giảm thiểu đáng kể chi phí sửa chữa và duy trì một nền tảng mã nguồn luôn ở trạng thái sẵn sàng phát hành.

d. Giai đoạn 4: Đóng gói phiên bản (Release)

Sau khi vượt qua các bài kiểm thử, mã nguồn được chuẩn bị để chuyển đổi thành một thực thể sẵn sàng phát hành chính thức.

- **Định danh và Dán nhãn:** Sản phẩm được gắn các nhãn phiên bản cụ thể (ví dụ: *Version 1.0.1*) để đảm bảo tính truy xuất nguồn gốc. Việc này giúp đội ngũ quản lý chính xác mã nguồn nào đang tương ứng với bản build nào trong hệ thống.
- **Chuẩn hóa môi trường (Containerization):** Sử dụng các công nghệ như *Docker* để đóng gói ứng dụng cùng toàn bộ cấu hình và thư viện phụ thuộc vào một *Container*. Quá trình này đảm bảo tính nhất quán tuyệt đối, loại bỏ sai lệch về môi trường khi di chuyển giữa các máy chủ khác nhau.
- **Lưu trữ tập trung:** Bản đóng gói cuối cùng được đẩy vào kho lưu trữ an toàn (*Registry*). Đây là nguồn duy nhất để triển khai lên môi trường thực tế, đảm bảo rằng mã nguồn đã được kiểm thử chính là mã nguồn được phân phối tới người dùng [7, 5].

e. Giai đoạn 5: Triển khai và Giám sát (Deploy & Monitor)

Giai đoạn cuối cùng của trực tuyến tính thực hiện việc đưa sản phẩm đến tay người dùng cuối và duy trì sự ổn định sau phát hành.

- **Chiến lược triển khai linh hoạt:** Áp dụng các kỹ thuật như *Blue-Green Deployment* hoặc *Canary Deployment* để chuyển đổi lưu lượng truy cập dần dần. Điều này giúp giảm thiểu rủi ro và cho phép kiểm tra thực tế trên một nhóm nhỏ người dùng trước khi phát hành toàn diện [5].
- **Tự động hóa triển khai:** Loại bỏ các thao tác thủ công dễ gây sai sót bằng các kịch bản triển khai tự động (*Infrastructure as Code*). Mục tiêu là biến việc phát hành phần mềm trở thành một quy trình lặp lại được và không gây căng thẳng cho đội ngũ vận hành [5].

- **Giám sát và Phản hồi (Monitoring):** Hệ thống liên tục thu thập dữ liệu về hiệu năng và lỗi sau triển khai. Nếu phát hiện bất thường, cơ chế *Rollback* tự động sẽ được kích hoạt để đưa hệ thống về trạng thái ổn định gần nhất, đảm bảo tính liên tục của dịch vụ.

3. Các chiến lược triển khai không gián đoạn (Zero-Downtime Deployment - ZDD)

Trong môi trường phần mềm hiện đại, việc đảm bảo tính sẵn sàng cao (High Availability) trong quá trình cập nhật phiên bản mới là yêu cầu bắt buộc. Zero-Downtime Deployment (ZDD) là tập hợp các chiến lược triển khai giúp hệ thống duy trì hoạt động liên tục, không làm gián đoạn trải nghiệm của người dùng cuối trong lúc nâng cấp [1].

a. Chiến lược Blue-Green Deployment

Blue-Green Deployment là chiến lược sử dụng hai môi trường sản xuất (production) độc lập, có cấu hình phần cứng và phần mềm giống hệt nhau, được gọi là Blue (môi trường hiện tại) và Green (môi trường mới). Tại một thời điểm, chỉ có duy nhất một môi trường phục vụ lưu lượng truy cập từ người dùng (live traffic) [8].

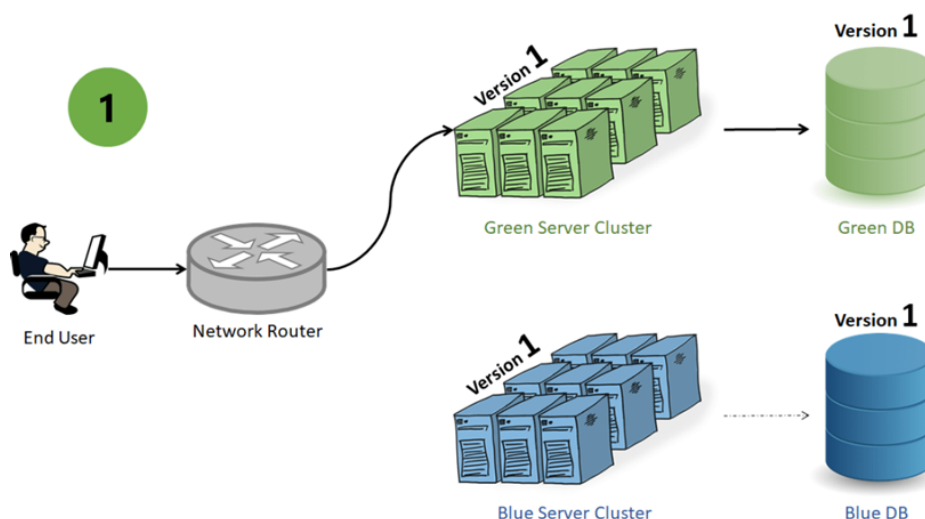


Figure 7: Mô hình kiến trúc chuyển hướng luồng truy cập trong Blue-Green Deployment

- **Cơ chế hoạt động và Vận hành:** Quá trình triển khai bắt đầu bằng việc đóng gói phiên bản mới và đẩy toàn bộ lên môi trường Green đang ở trạng thái rảnh rỗi (idle). Tại đây, đội ngũ QA và hệ thống kiểm thử tự động có thể thực hiện kiểm thử chấp nhận (User Acceptance Testing - UAT) hoặc kiểm thử tải (Load Testing) trên một môi trường giống hệt môi trường thực tế mà không lo ảnh hưởng đến người dùng. Khi mọi bài kiểm tra đều vượt qua, một thao tác "chuyển mạch" (switchover) sẽ được thực hiện trên cấu hình của Router. Toàn bộ lưu lượng truy cập sẽ được điều hướng tức thì từ Blue sang Green. Môi trường Blue lúc này sẽ chuyển sang trạng thái rảnh rỗi và đóng vai trò như một bản sao lưu (backup) khẩn cấp.
- **Ưu điểm:** Ưu điểm lớn nhất của mô hình này là thời gian gián đoạn (downtime) gần như bằng không. Nếu phiên bản Green phát sinh lỗi nghiêm trọng sau khi nhận traffic thật, khả năng phục hồi (rollback) diễn ra cực kỳ nhanh chóng và an toàn chỉ bằng việc cấu

hình lại Router trả luồng mạng về môi trường Blue. Tuy nhiên, thách thức lớn nhất của Blue-Green không nằm ở chi phí nhân đôi hạ tầng, mà nằm ở bài toán đồng bộ hóa cơ sở dữ liệu (Database Migration). Việc hai môi trường chạy hai phiên bản mã nguồn khác nhau nhưng phải truy xuất chung một cơ sở dữ liệu đòi hỏi các kỹ sư phải áp dụng các mẫu thiết kế phức tạp như Mở rộng và Thu hẹp (Expand and Contract Pattern) để tránh xung đột lược đồ (schema) [5].

Ví dụ: Vấn đề đồng bộ cơ sở dữ liệu trong chiến lược Blue-Green từng là rào cản rất lớn cho các hệ thống khổng lồ. Nhận thấy điều này, vào cuối năm 2022, nền tảng điện toán đám mây Amazon Web Services (AWS) đã chính thức ra mắt tính năng *Amazon RDS Blue/Green Deployments* [13]. Trước sự kiện này, việc cập nhật cơ sở dữ liệu mà không làm gián đoạn hệ thống đòi hỏi quá trình lập kế hoạch phức tạp. Tuy nhiên, với giải pháp của AWS ra mắt năm 2022, hệ thống tự động tạo ra một bản sao (Green) của cơ sở dữ liệu và đồng bộ hóa logic theo thời gian thực với cơ sở dữ liệu chính (Blue). Khi nâng cấp hoàn tất, AWS sử dụng các rào chắn bảo vệ (guardrails) để chuyển đổi luồng dữ liệu chỉ trong chưa tới một phút, không làm rớt bất kỳ giao dịch nào. Điều này đã mang mô hình Blue-Green từ lý thuyết hạ tầng mạng áp dụng thành công xuống tầng dữ liệu.

b. Chiến lược Canary Release

Canary Release (Phát hành chim hoàng yến) là kỹ thuật triển khai phiên bản phần mềm mới cho một nhóm rất nhỏ người dùng thực tế trước, sau đó mới tiến hành mở rộng dần ra toàn bộ hệ thống. Thuật ngữ này lấy cảm hứng từ lịch sử ngành khai thác mỏ thế kỷ 20, khi những người thợ mỏ thường mang theo một con chim hoàng yến vào hầm. Vì loài chim này rất nhạy cảm với khí độc, nếu chim có dấu hiệu bất thường, thợ mỏ sẽ biết để sơ tán kịp thời [9]. Trong phần mềm, nhóm người dùng đầu tiên trải nghiệm tính năng mới chính là những "chú chim hoàng yến" giúp phát hiện lỗi hệ thống sớm nhất.

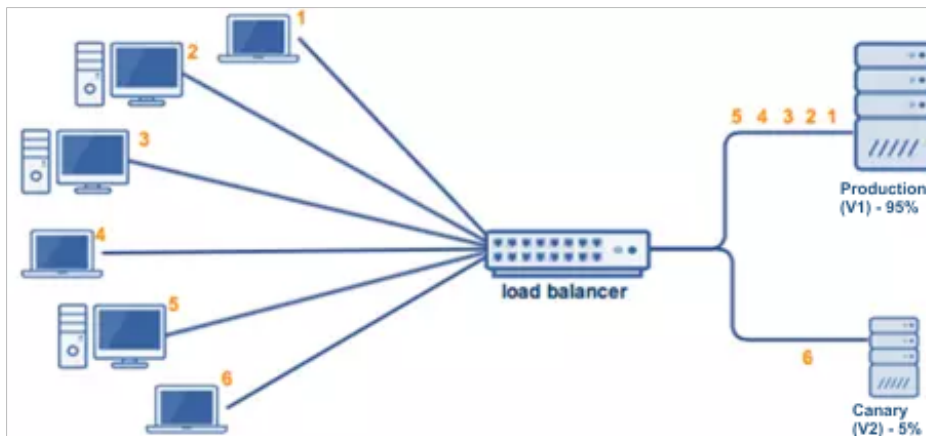


Figure 8: Mô hình phân luồng lưu lượng truy cập trong Canary Release

- **Cơ chế hoạt động và Đo lường:** Khác với sự chuyển đổi toàn bộ của Blue-Green, Canary Release hoạt động dựa trên cơ sở định tuyến phân mảnh. Khi phiên bản mới được phát hành, hệ thống điều hướng (như API Gateway hoặc Service Mesh) sẽ chỉ đẩy một tỷ lệ lưu lượng cực nhỏ (ví dụ: 1% hoặc 5% người dùng) sang các máy chủ mới. Đội ngũ vận hành sẽ thiết lập một khoảng thời gian chờ (bake time) để theo dõi chặt chẽ các chỉ số hiệu năng (metrics) như thời gian phản hồi (latency), số lượng mã lỗi HTTP 500, hay mức tiêu thụ CPU. Nếu hệ thống duy trì sự ổn định, tỷ lệ định tuyến này sẽ được tăng dần (từ 10% lên 25%, 50% và cuối cùng là 100%) cho đến khi phiên bản cũ được thay thế hoàn toàn.

- **Ưu điểm và Điểm yếu cốt lõi:** Chiến lược này cung cấp mức độ an toàn cao nhất nhờ việc giới hạn tối đa "bán kính ảnh hưởng" (blast radius). Nếu phiên bản mới chứa một lỗi hỏng nghiêm trọng làm sập hệ thống, chỉ 1% người dùng bị ảnh hưởng thay vì toàn bộ khách hàng. Tuy nhiên, nhược điểm cốt lõi là quá trình triển khai có thể kéo dài hàng giờ hoặc hàng ngày. Ngoài ra, Canary yêu cầu một hệ thống giám sát (observability) và phân tích nhật ký (logging) cực kỳ tinh vi để có thể so sánh tự động hành vi giữa nhóm người dùng cũ và mới [1].

Ví dụ: Vào tháng 4 năm 2018, Netflix phối hợp cùng Google chính thức phát hành công cụ nguồn mở mang tên *Kayenta* [6], một nền tảng Phân tích Canary Tự động (Automated Canary Analysis - ACA). Netflix lúc bấy giờ đang triển khai hàng ngàn bản cập nhật mỗi ngày trên kiến trúc Microservices khổng lồ. Việc để kỹ sư tự phân tích biểu đồ giám sát của 1% traffic là điều bất khả thi và dễ sai sót. Việc ra mắt Kayenta vào năm 2018 đã thay đổi hoàn toàn cục diện: hệ thống tự động thu thập hàng ngàn chỉ số từ cụm Canary và cụm Baseline (phiên bản cũ), sử dụng các thuật toán thống kê để đưa ra điểm số tin cậy. Nếu điểm số này giảm xuống dưới mức cho phép, Kayenta sẽ tự động hủy phiên bản Canary và rollback hệ thống mà không cần con người can thiệp.

c. Chiến lược Rolling Update

Rolling Update (Cập nhật cuốn chiếu) là phương pháp triển khai tập trung vào việc cập nhật dần dần từng cụm máy chủ (instance hoặc node) của hệ thống, thay vì phải nhân đôi toàn bộ hạ tầng (như Blue-Green) hoặc chia nhỏ lưu lượng người dùng dựa trên bộ định tuyến (như Canary). Chiến lược này đảm bảo số lượng máy chủ phục vụ ứng dụng luôn được duy trì ở mức tối thiểu để xử lý tải, tối ưu hóa được chi phí tài nguyên phần cứng [14].

- **Cơ chế hoạt động và Điều phối tự động:** Quá trình Rolling Update diễn ra theo một vòng lặp liên tục. Hệ thống điều phối sẽ tạm thời gỡ bỏ một nhóm nhỏ các máy chủ đang chạy phiên bản cũ khỏi bộ cân bằng tải. Tiếp theo, nó sẽ triển khai phiên bản mới lên các máy chủ vừa được gỡ ra, khởi động lại chúng và chờ cho đến khi ứng dụng thực sự sẵn sàng nhận kết nối. Khi máy chủ mới báo trạng thái ổn định, nó sẽ được gắn lại vào hệ thống mạng. Quá trình "tháo ra - cập nhật - gắn vào" này sẽ lặp đi lặp lại (cuốn chiếu) cho đến khi 100% các máy chủ trong cụm đều mang phiên bản mới.
- **Ưu điểm và Lưu ý vận hành:** Điểm mạnh nhất của Rolling Update là tính kinh tế, không đòi hỏi hệ thống tồn kém tài nguyên gấp đôi. Sự chuyển giao diễn ra mượt mà giúp phân bổ đều tải trọng mạng, tránh tình trạng giật lag đột ngột. Dù vậy, điểm yếu của nó là tốc độ khôi phục (rollback) khá chậm, bởi hệ thống phải thực hiện lại thao tác cuốn chiếu theo chiều ngược lại. Đặc biệt, vì người dùng có thể truy cập ngẫu nhiên vào cả máy chủ cũ và mới trong suốt quá trình cập nhật, ứng dụng bắt buộc phải được thiết kế với khả năng tương thích ngược (backward compatibility) chặt chẽ đối với các hàm API và cơ sở dữ liệu [5].

Ví dụ: Sự kiện mang tính bước ngoặt đưa chiến lược này trở thành tiêu chuẩn toàn cầu là sự phát triển của nền tảng *Kubernetes*. Vào tháng 3 năm 2016, Kubernetes tung ra phiên bản 1.2, chính thức đưa tài nguyên điều phối Deployment vào sử dụng thực tiễn và biến Rolling Update thành chiến lược mặc định (Default Deployment Strategy) [10]. Trước năm 2016, để làm được điều này, các công ty phải tự viết những kịch bản script dài hàng ngàn dòng, cực kỳ dễ đứt gãy. Với bản cập nhật năm 2016 của Kubernetes, kỹ sư giờ đây chỉ cần khai báo hai thông số đơn giản là `maxSurge` (số máy chủ tối đa được phép vượt ngưỡng) và `maxUnavailable` (số máy chủ tối đa được phép ngừng hoạt động). Tính năng này đã tự động hóa hoàn toàn bài toán nâng cấp không gián đoạn cho hàng triệu dự án lớn nhỏ ngày nay.

d. Bảng so sánh các chiến lược

Dưới đây là bảng so sánh tổng quan đặc điểm của 3 chiến lược ZDD phổ biến, hỗ trợ đội ngũ phát triển lựa chọn phương pháp phù hợp với kiến trúc dự án:

Table 1: So sánh các chiến lược triển khai không gián đoạn

Tiêu chí	Blue-Green	Canary Release	Rolling Update
Mức độ rủi ro	Thấp (Rollback tức thì)	Rất thấp (Chỉ ảnh hưởng nhóm nhỏ)	Trung bình (Phải tương thích 2 phiên bản)
Chi phí hạ tầng	Rất cao (x2 tài nguyên)	Vừa phải (Thêm tài nguyên nhỏ ban đầu)	Thấp (Tận dụng cụm server hiện có)
Tốc độ Rollback	Rất nhanh (Đổi route)	Nhanh	Chậm (Phải update ngược lại)
Độ phức tạp	Trung bình	Cao (Cần hệ thống monitor & routing mạnh)	Thấp (Được hỗ trợ sẵn bởi K8s, Docker)
Use Case lý tưởng	Hệ thống cốt lõi, cần rollback nhanh, server dư dả.	Thử nghiệm tính năng, có hệ thống theo dõi tốt.	Cập nhật thường xuyên, tài nguyên giới hạn.

IV. MÔ HÌNH HÓA QUY TRÌNH CI/CD CỦA DEVOPS

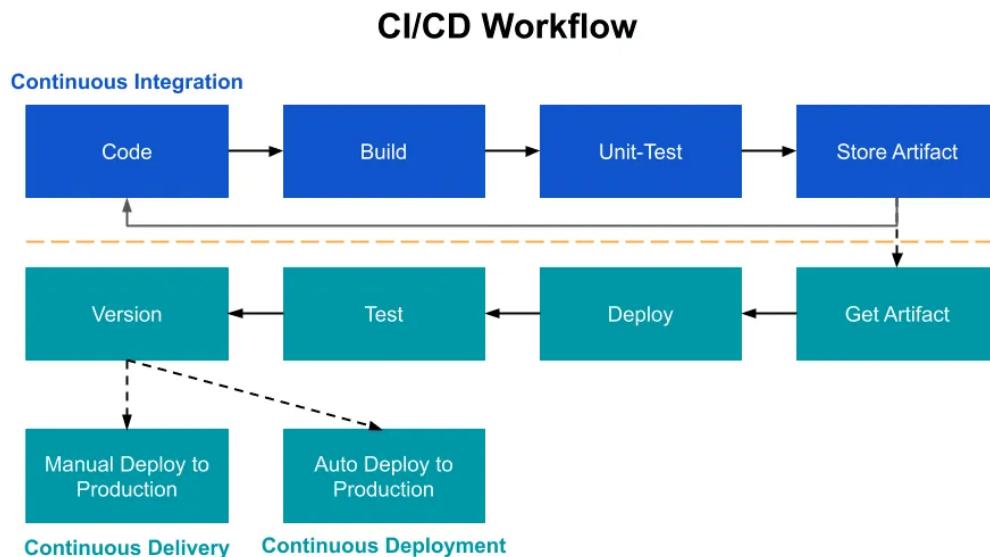


Figure 9: Mô hình hóa luồng công việc CI/CD và các chiến lược triển khai thành phẩm

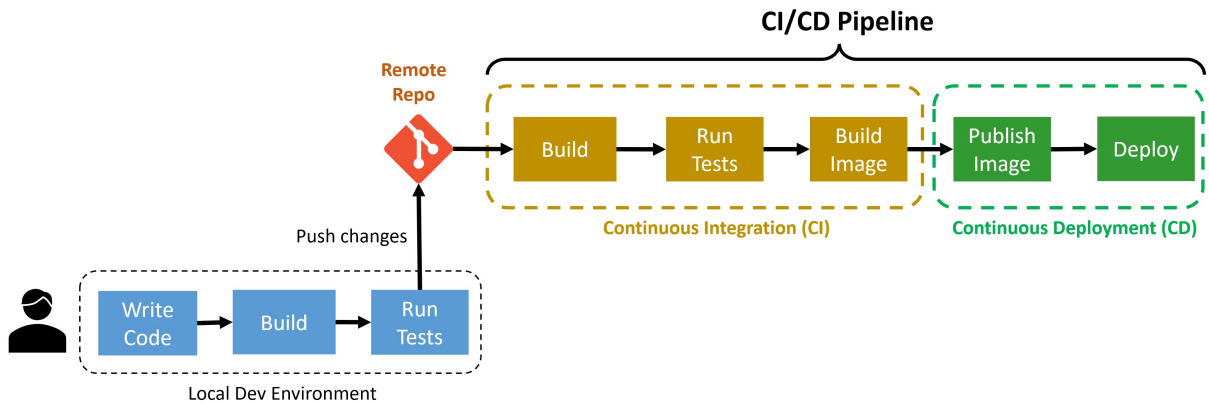


Figure 10: Luồng dữ liệu từ môi trường phát triển cục bộ đến triển khai thực tế qua Pipeline

V. KẾT LUẬN

1. Thành tựu đạt được

Thông qua quá trình nghiên cứu và thực hiện đề tài về mô hình hóa quy trình CI/CD của DevOps, nhóm đã hoàn thành việc xây dựng một hệ thống lý thuyết và thực tiễn vững chắc, góp phần thu hẹp khoảng cách giữa lý thuyết mô hình hóa thuần túy và thực tiễn tự động hóa hiện đại. Thành tựu quan trọng nhất chính là việc áp dụng thành công bộ tiêu chuẩn UML 2.x để đặc tả hình thức cho một hệ thống phần mềm phân tán cực kỳ phức tạp như đường ống CI/CD. Nhóm đã trực quan hóa được các luồng dữ liệu và luồng điều khiển giữa các giai đoạn từ lúc mã nguồn được đẩy lên hệ thống quản lý phiên bản cho đến khi tạo ra sản phẩm thực thi cuối cùng.

Việc sử dụng các sơ đồ hoạt động và sơ đồ trình tự đã giúp làm rõ các "chốt chặn" kiểm soát chất lượng cũng như cơ chế phản hồi vòng lặp khi các bài kiểm thử thất bại, điều mà các tài liệu văn bản thông thường khó có thể mô tả chi tiết[cite: 59, 204]. Bên cạnh đó, nhóm đã mô hình hóa thành công trạng thái và luồng hoạt động của ba chiến lược triển khai không gián đoạn kinh điển bao gồm Blue-Green Deployment, Canary Release và Rolling Update. Thành công này không chỉ dừng lại ở mức độ lý thuyết mà còn đạt được sự cân bằng tối ưu trong việc trừu tượng hóa: các mô hình được xây dựng đảm bảo tính độc lập với công nghệ, không phụ thuộc vào các công cụ cụ thể như Jenkins hay GitLab CI, nhưng vẫn đủ chi tiết để đóng vai trò làm bản thiết kế kỹ thuật cho việc triển khai thực tế.

2. Hạn chế của đề tài

Mặc dù đã đạt được những mục tiêu cốt lõi, bài báo cáo vẫn tồn tại những hạn chế nhất định xuất phát từ giới hạn thời gian và kinh nghiệm thực tiễn đối với các hệ thống ở quy mô công nghiệp lớn. Hạn chế lớn nhất nằm ở tính động của hệ thống thực tế so với tính tĩnh của các ngôn ngữ mô hình hóa truyền thống. Các quy trình CI/CD hiện đại thường mang tính trạng thái cao và sở hữu các logic xử lý ngoại lệ vô cùng phức tạp mà việc sử dụng các sơ đồ UML đôi khi chưa thể phản ánh một cách trọn vẹn mà không làm cho sơ đồ trở nên rối rắm.

Về mặt phạm vi, đề tài mới chỉ tập trung phân tích ba chiến lược triển khai phổ biến nhất hiện nay, do đó chưa thể mở rộng sang các kịch bản lai ghép hoặc các chiến lược nâng cao như Shadow Deployment hay A/B Testing. Ngoài ra, mặc dù đã đề cập đến các mẫu thiết kế để giải quyết vấn đề đồng bộ hóa cơ sở dữ liệu như mẫu Mở rộng và Thu hẹp, việc mô hình hóa chi tiết sự thay đổi lược đồ dữ liệu trong môi trường phân tán vẫn chỉ dừng lại ở mức độ khái quát hóa, chưa đi sâu vào các ràng buộc toàn vẹn dữ liệu ở mức độ vi mô. Cuối cùng, việc lựa chọn UML

làm ngôn ngữ chính dù đảm bảo tính chuẩn hóa nhưng đôi khi lại quá nặng nề so với các nhu cầu đặc tả quy trình nghiệp vụ nhanh trong các dự án nhỏ.

3. Hướng phát triển trong tương lai

Để khắc phục những hạn chế nêu trên và bắt kịp với tốc độ phát triển của ngành công nghiệp phần mềm, nhóm đề xuất các hướng phát triển tiếp theo nhằm nâng cao giá trị học thuật và thực tiễn của đề tài. Một trong những hướng đi ưu tiên là tích hợp tư duy bảo mật vào mô hình thông qua việc nghiên cứu DevSecOps, nơi các giai đoạn quét mã tĩnh và kiểm tra lỗ hổng động sẽ được mô hình hóa như những thành phần không thể tách rời của pipeline.

Sự phát triển của trí tuệ nhân tạo cũng mở ra cơ hội cho việc mô hình hóa các hệ thống CI/CD dựa trên AI, cụ thể là việc tích hợp các thuật toán thống kê và học máy để tự động hóa khâu phân tích Canary giống như mô hình Kayenta mà Netflix đã triển khai thành công. Trong tương lai, nhóm hướng tới việc ứng dụng các ngôn ngữ mô hình hóa có tính toán học cao hơn như Petri Nets để mô phỏng chính xác các hành vi song song và tranh chấp tài nguyên trong hạ tầng Cloud-native. Mục tiêu cuối cùng là hướng tới một quy trình "Model-Driven DevOps", nơi mà từ các mô hình UML đã được thiết kế và kiểm thử logic, chúng ta có thể tự động sinh ra các mã cấu hình thực tế, giúp tối ưu hóa hoàn toàn hiệu suất bàn giao phần mềm và giảm thiểu sai sót do con người gây ra.

TÀI LIỆU THAM KHẢO

- [1] G. Kim, J. Humble, P. Debois, and J. Willis, *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. Portland, OR: IT Revolution Press, 2016.
- [2] G. Kim, K. Behr, and G. Spafford, *The Phoenix Project: A Novel About IT, DevOps, and Helping Your Business Win*. Portland, OR: IT Revolution Press, 2013.
- [3] J. Davis and R. Daniels, *Effective DevOps: Building a Culture of Collaboration, Affinity, and Tooling at Scale*. Sebastopol, CA: O'Reilly Media, Inc., 2016.
- [4] V. T. Kiệt, “Slide bài giảng môn DevOps trong phát triển phần mềm.” Khoa Công nghệ Phần mềm, Trường Đại học Công nghệ Thông tin - ĐHQG TP.HCM, 2026.
- [5] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison-Wesley Professional, 2010.
- [6] N. T. Blog, “Automated canary analysis at netflix with kayenta.” <https://netflixtechblog.com/automated-canary-analysis-at-netflix-with-kayenta-3260bc7acc69>, 2018.
- [7] M. Fowler, “Continuous integration.” <https://martinfowler.com/articles/continuousIntegration.html>, 2006.
- [8] M. Fowler, “Bluegreendeployment.” <https://martinfowler.com/bliki/BlueGreenDeployment.html>, 2010.
- [9] D. Sato, “Canaryrelease.” <https://martinfowler.com/bliki/CanaryRelease.html>, 2014.
- [10] Kubernetes Authors, “Kubernetes 1.2 and simplifying advanced networking with ingress.” <https://kubernetes.io/blog/2016/03/kubernetes-1-2-and-simplifying-advanced-networking-with-ingress/>, 2016.
- [11] J. Rumbaugh, I. Jacobson, and G. Booch, *The Unified Modeling Language Reference Manual*. Pearson Higher Education, 2004.
- [12] M. Fowler, “Continuous delivery.” <https://martinfowler.com/books/continuousDelivery.html>, 2010.
- [13] Amazon Web Services, “New – amazon rds blue/green deployments for safer, simpler, and faster updates.” <https://aws.amazon.com/blogs/aws/new-amazon-rds-blue-green-deployments-for-database-updates/>, 2022.
- [14] Kubernetes Authors, “Deployments.” <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>, 2026.